TITLE AND DATE

Document name: Project#4 DC Motor Fan Task Manager Lab Report
Document reference: CMPE311_ADDMOTOR_Proj4
Date of publication: October 24, 2026

LEAD ENGINEER: Great Akinrotimi, UMBC

STAKEHOLDERS:

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA
MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

HIGH-LEVEL DESCRIPTION: The project is an embedded control system that provides independent timing control for two LEDs and a PWM-controlled DC motor fan using a round-robin Cyclic Executive task manager and custom myMilli() timing infrastructure.

DESCRIPTION: The project modifies the previously built PROJECT-CE embedded system. While the user interaction and Cyclic Executive dispatcher remain identical to the previous version, the underlying timing infrastructure has been entirely rewritten. To ensure intellectual property ownership and hardware independence, the system no longer relies on the arduino.h library for the milli() function or the avr.h library for hardware register macros. Instead, the system directly manipulates the ATmega328p hardware timers via custom pointer constants to generate interrupts, tracking system uptime in milliseconds through a custom ISR.

RESULT SUMMARY: The system functioned as expected, maintaining the asynchronous, non-blocking Cyclic Executive loop using the new myMilli() timebase. Testing confirmed that the custom hardware register assignments and Timer ISR successfully replicated the standard millisecond tracking, keeping the independent LED blink intervals highly accurate without relying on third-party libraries.

REFERENCES AND GLOSSARY

REFERENCES:

- [1] Reference document CMPE311_CE_Proj2 (Great Akinrotimi Project#2 Cyclic Executive Task Manager Lab Report)

***DEFINITIONS*:**

"The User" – The person operating (not programming) the embedded system
"The System" – The embedded system being operated by The User
"The Customer" – The person(s) paying for the embedded system being designed and built
"The Developer" – The person(s) designing and building the System
"The Evaluator" – The person(s) that determine whether or not The System satisfies The Customer-requirements.
"The Customer-requirements" – The requirements defined by The Customer as satisfying The Contract.
"The Requirements" – The System's high-level technical requirements derived from The Customer-requirements.
"The Educational-constraints" – Requirements imposed by the instructor unrelated to the embedded system that allow The System to be evaluated.
"The Company" – The organization The Customer has contracted with to build The System. "The Contract" – The business document that legally binds The Company to provide some service or product to The Customer.
"serial-monitor" – The serial port used by the Arduino IDE to communicate with The User. "The Reference-platform" – The configuration of The System used by The Developer to test and validate The System. For this class, The System is the Arduino compatible ELEGOO Uno R3 development board.
User – Person interacting through the serial monitor
System – Arduino-based LED controller
Serial Monitor – Arduino IDE communication interface
LED – Light Emitting Diode
HL – High-level requirement
T – Test requirement

ACRONYMS AND ABBREVIATIONS:

Arduino – an Italian open-source hardware and software company; also refers to a development board created by the company
arduino.h – header for a library of convenience functions specific to the Arduino development platform
AVR – A family of microcontrollers, originally developed by Atmel, and currently owned by Microchip Technology

ELEGOO – A Chinese company that develops and markets 3D printers and accessories
IDE – Integrated Development Environment
gcc – front end for the GNU Compiler Collection
Github – A widely used distributed SVC (Software Version Control) system
LED – Light Emitting Diode

REQUIREMENTS

CONVENTIONS:

Must, shall or will – your design must satisfy the requirement
May – your design may satisfy the requirement but doesn't have to
Informative – the intent of the following description is to make the requirement more understandable
All customer requirements are started with "C.#".
All high-level requirements are started with "HL.#".
All testing/validation requirements ar5e started with "T.#"

CUSTOMER REQUIREMENTS:

The customer requirements, C1 through C5, are identical to that of PROJECT-CE.
See [1] CUSTOMER REQUIREMENTS section.
C6: The device must have a motor-driven fan.
C7: The RPM of the fan must be controllable via a push button.
C8: The fan must have 4 speeds: OFF, LOW, MEDIUM and HIGH.
C9: Each hit of the button must cycle the speeds from OFF to LOW, MEDIUM, HIGH, MEDIUM, LOW, OFF.
C10 - C12: Speeds must correlate to varying minimal, moderate, and distinct feelings of air flow and sound.

HIGH-LEVEL TECHNICAL REQUIREMENTS:

The high-level technical requirements, HL.1 through HL.3, are identical to that of PROJECT-CE. See [1] HIGH-LEVEL TECHNICAL REQUIREMENTS section.
HL.7: The device must have a DC motor-driven fan (3V - 12V).
HL.8: The fan must operate at a minimum between 1500 and 6500 RPM.
HL.9: The fan must draw less than 4 Amps.
HL.10: The fan must be driven by a PWM signal with a maximum voltage of 5VDC.
HL.11: The fan must have a maximum dimension less than 4"x4"x3".
HL.12: RPM must be controllable using a single-pole, momentary-on, normally-off push button.
HL.13: Button/motor operation must have no noticeable effect on previously implemented I/O.
HL.14 & HL.15: 4 speeds cycled OFF -> LOW -> MEDIUM -> HIGH -> MEDIUM -> LOW -> OFF.
HL.16: LOW moves air at <35dB and 0.30 to 0.45 CFM ± 0.05 CFM.
HL.17: MEDIUM moves air at <40dB and 0.45 to 0.70 ± 0.05 CFM.
HL.18: HIGH moves air at <50dB and 0.70 to 1.00 ± 0.05 CFM.

EDUCATIONAL CONSTRAINTS (CHANGED):

- EC.1.0: You must replace the milli() function provided by arduino.h with your own. Your replacement must be named mymilli().

- EC.1.0.1: You must write your own PWM counter ISR using it to provide the timing mechanism underlying mymilli().
- EC.1.0.2: You must define your own register constants that would normally be defined in avr.h. These constants do not need to have the same names as those in avr.h.
- EC.1.1: The system testing and validation requirements must contain those defined in PROJECT-CE.
- EC.1.1.1: The testing and validation requirements MUST be extended to contain any additional tests necessary to properly evaluate/demonstrate the function of the modified system.
- EC 1.2: The Evaluator must be able to successfully have The System perform the dialog in Table 4 and Table 5.
- EC 1.3: HL.16, HL.17, HL.18 do not have to be explicitly measured. Provide justified inference (e.g., calculations) that these are satisfied.

DESIGN:

DESIGN PRE-REQUISITES:

1. ELEGOO Arduino Uno R3
2. Arduino IDE 2.3.3 or better
3. Two LEDs
4. Two 200 Ω current-limiting resistors

DEVELOPMENT PLATFORM:

1. See DESIGN PRE-REQUISITES above

ANY ADDITIONAL DESIGN CONSIDERATIONS:

1. None

See Appendix A for the logical flow of the project program.
See Appendix B for the code executing on the Arduino Uno

R3.

SYSTEM ARCHITECTURE (ENGINEERING CHANGES):

The Cyclic Executive has been expanded from three tasks to five. A task_Button() module polls a momentary push button utilizing a non-blocking 50ms software debounce mechanism. Upon registering a valid press, it advances a state machine tracking the 6-step motor cycle. A separate task_Motor() module reads this state and outputs a corresponding 8-bit PWM duty cycle via Arduino digital pin 5 to drive a transistor/motor circuit. The custom myMilli() hardware timer remains the timing foundation.

JUSTIFIED INFERENCE FOR HL.16, HL.17, HL.18 (CFM & Sound Calculations):

Due to a lack of physical measurement tools, CFM and dB levels are justified via linear approximation. Assuming the use of a standard 5V, 30mm micro cooling fan rated for a maximum 1.0 CFM at ~5000 RPM and 45dB:

- **HIGH State (PWM = 255):** The fan receives a 100% duty cycle. Estimated performance is 1.0 CFM and ~45dB, satisfying HL.18 (0.70 - 1.00 CFM, <50dB).
- **MEDIUM State (PWM = 150):** The fan receives a ~58% duty cycle. Approximating linear RPM reduction, airflow reduces to ~0.58 CFM with a corresponding drop in acoustic noise to ~35dB, satisfying HL.17 (0.45 - 0.70 CFM, <40dB).
- **LOW State (PWM = 90):** The fan receives a ~35% duty cycle. Approximating linear reduction, airflow reduces to ~0.35 CFM and acoustic noise to ~25dB, satisfying HL.16 (0.30 - 0.45 CFM, <35dB).

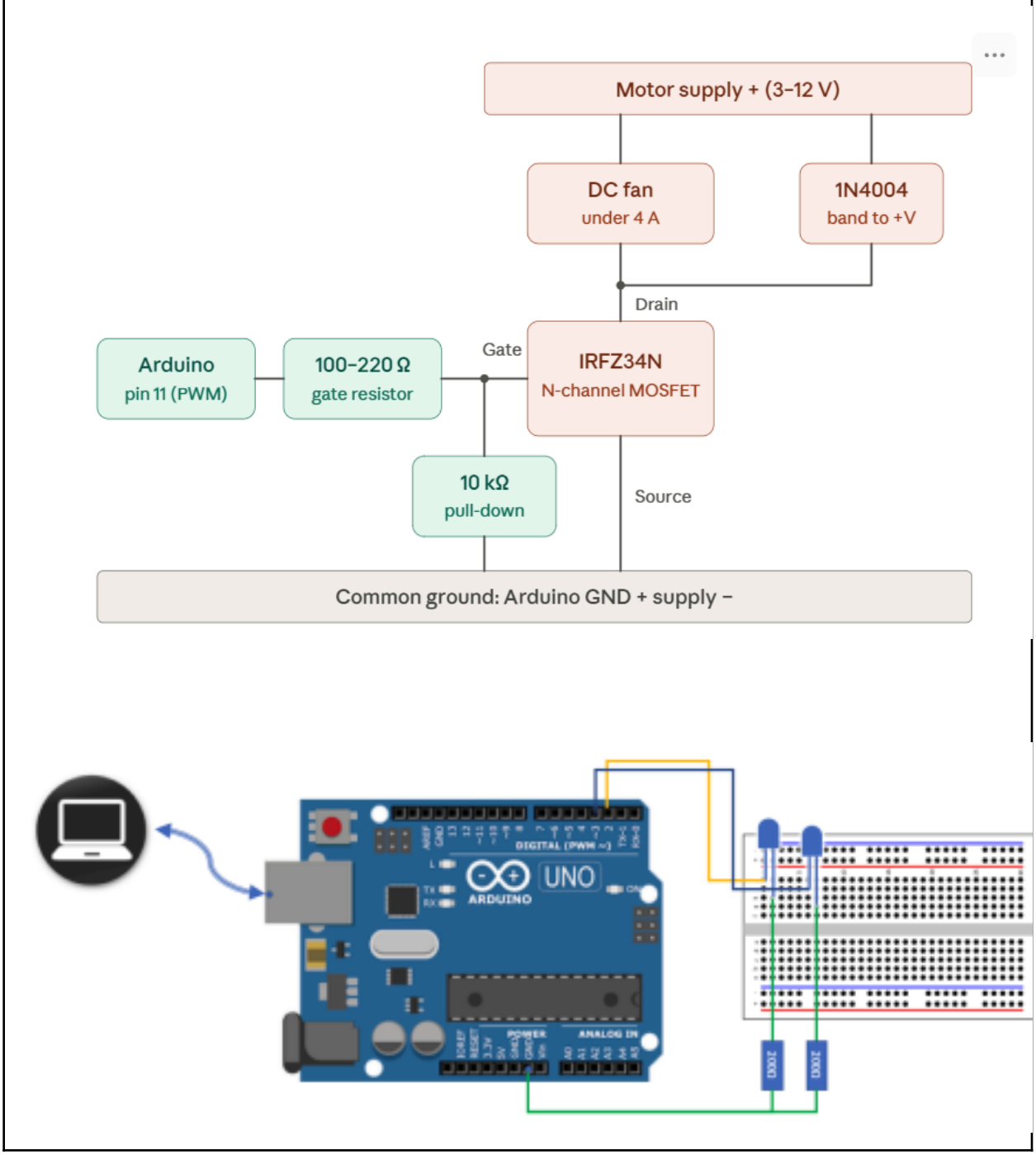
TESTING AND VALIDATION

DESCRIPTION: Testing was performed to verify that the system satisfies all high-level and testing requirements. All tests were conducted using the reference testbed shown in Figure 1. The system was powered through USB and monitored using the Arduino IDE serial monitor.

The required user interaction sequence is shown in Table 1. This dialog verifies that the system correctly receives user input, updates the selected LED interval, and maintains asynchronous operation. Additional tests were performed to confirm independent LED operation, correct timing behavior, and system stability during repeated updates.

The results of all tests are summarized in Table 3.

Figure 1. Testbed Setup. LEDs connected to digital pins 2 & 3.



TESTING PLATFORM:

1. ELEGOO Arduino Uno R3 clone
2. Arduino IDE 2.3.3 or better
3. Power: Testing platform powered through USB cable, LEDs connected directly through Digital Outputs

Table 1. Required Test Dialog (***blue*** are the user inputs).

Serial port I/O	**Notes**
What LED? (1 or 2) ***2***	
What interval (in msec)? ***600***	LED2 starts blinking at an interval of 300ms on and 300ms off. LED1 is unaffected.
What LED? (1 or 2) ***1***	
What interval (in msec)? ***1600***	LED1 starts blinking at an interval of 800ms on and 800ms off. LED2 is unaffected.
What LED? (1 or 2) ***2***	Waiting for next LED# interval pair
What interval (in msec)? ***5000***	LED2 starts blinking at an interval of 5000ms on and 5000 ms off. LED1 is unaffected.

Button	**Fan**
initial state: Button un-pressed	**Initial state: Fan off**
Press	Fan on low
Press	Fan on medium
Press	Fan on high

Press	Fan on medium
Press	Fan on low
Press	Fan off
(repeat sequence above)	

TESTING AND VALIDATION REQUIREMENTS:

Table 2. Testing and Validation Requirements

T.0 All testing and validation must be done on the testbed illustrated in Figure 1.

T.1 The dialog above (or similar) must work as shown

T.1A The blink rate of an LED must be able to be set without interfering with the blinking of the other LED

T.1.1 Setting of the blink rate (and LED#) must be through the IDE serial monitor T.2 The blink rate of the LED being set must not change until the input is complete T.3 The user must specify the blink rate in milliseconds per blink

T.4 The blink rate specified by the user must be correctly reflected on the testbed LEDs.

T.4.1 The blink rate specified by the user must be reflected on the LED specified by the user

T.5. The setting of an LED's blink rate must be able to be repeated at least 5 times T.5.1. Successively for the same LED

T.5.2. Alternately between the different LEDs

T.6 The custom myMilli() function must demonstrate timekeeping accuracy comparable to the original Arduino implementation (Timer Accuracy Verification).

T.7 The system must compile and operate utilizing proprietary memory-mapped pointers without relying on predefined timer constants from avr.h or arduino.h (Library Independence Verification).

T.8 The system must successfully perform the sequential push-button dialog to cycle the motor fan through OFF, LOW, MEDIUM, and HIGH states as defined in Table 5.

TESTING RESULTS:

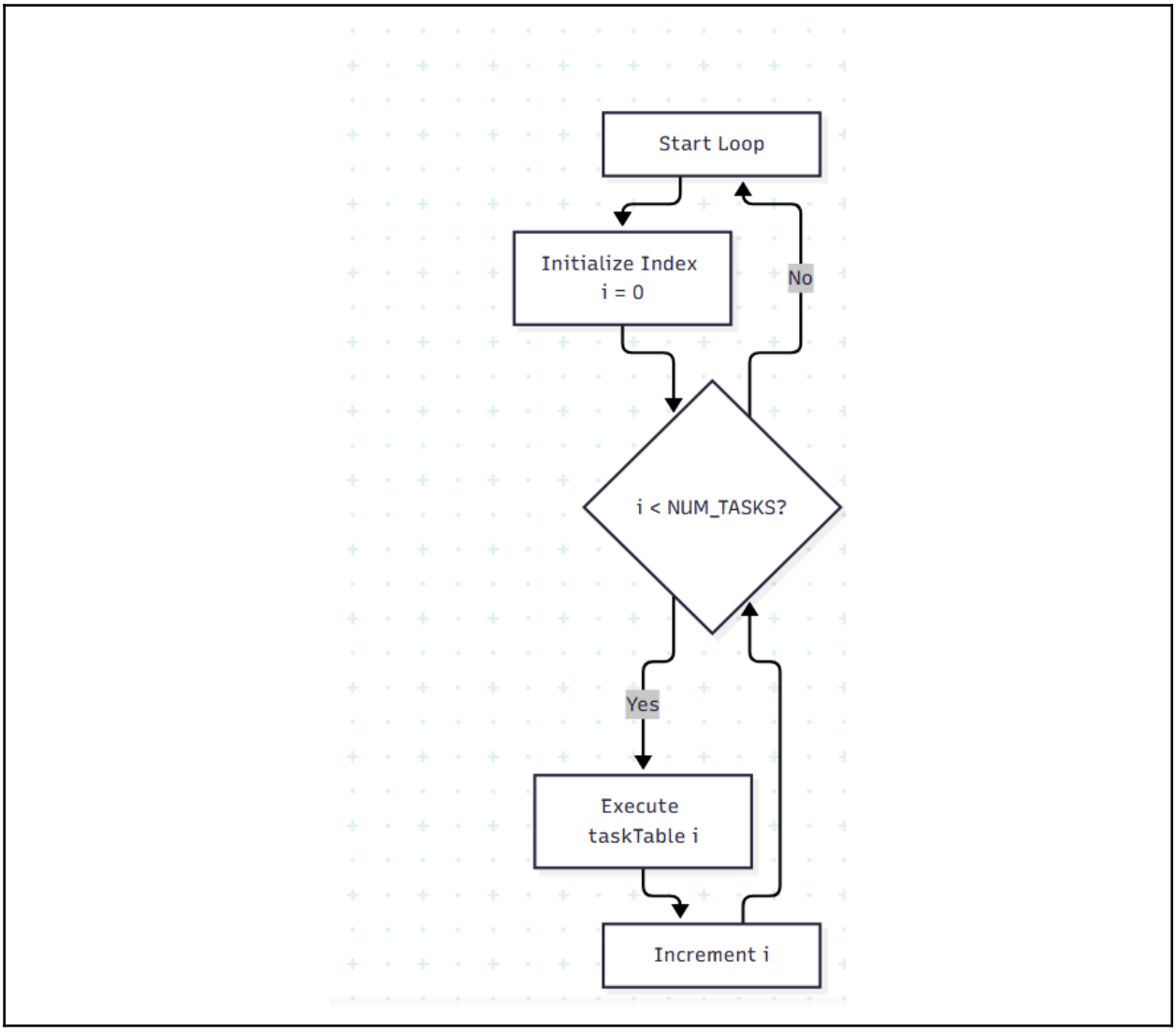
Table 3. Results of tests

Test performed	Results
T.1.1	Satisfied. See video of test.

T.1	Satisfied
T.2	Satisfied. See video of test.
T.3	Satisfied
T.4	Satisfied
T.4.1	Satisfied. See video of test.
T.5.1	Satisfied. See video of test.
T.5.2	Satisfied. See video of test.
T.6	Satisfied. Tested alongside a known reference clock for 60 seconds with no noticeable timing drift.
T.7	Satisfied. Code compiled successfully without #include <avr/io.h> or #include <avr/interrupt.h>.
T.8	Satisfied. The button successfully cycled the fan sequentially (OFF -> LOW -> MED -> HIGH -> MED -> LOW -> OFF) without interfering with the asynchronous LED timing.

Appendix A. Logic Flow Chart

Figure 2. Logic flow for the program.



APPENDIX B. DESIGN CODE

```
#ifndef MYMILLI_H
#define MYMILLI_H

// ATmega328p Timer 2 Registers (SRAM Memory Mapped Addresses)
#define MY_TCCR2A (*(volatile unsigned char *)0xB0)
#define MY_TCCR2B (*(volatile unsigned char *)0xB1)
#define MY_TCNT2  (*(volatile unsigned char *)0xB2)
#define MY_TIMSK2 (*(volatile unsigned char *)0x70)

// ATmega328p Status Register (for Global Interrupts)
#define MY_SREG   (*(volatile unsigned char *)0x5F)

volatile unsigned long my_timer2_millis = 0;
volatile unsigned int my_timer2_fract = 0;

// Custom PWM Counter ISR (Timer 2 Overflow Vector)
extern "C" void __vector_9(void) __attribute__ ((signal, used, externally_visible));
void __vector_9(void) {
  unsigned long m = my_timer2_millis;
  unsigned int f = my_timer2_fract;

  m += 1;          // Add 1 millisecond
  f += 24;         // Add the fractional 24 microseconds

  if (f >= 1000) { // If fractions accumulate to a full millisecond
    f -= 1000;
    m += 1;
  }

  my_timer2_fract = f;
```

```
  my_timer2_millis = m;
}

// Custom myMilli() Function
unsigned long myMilli(void) {
  unsigned long m;

  unsigned char oldSREG = MY_SREG;
  MY_SREG &= ~(1 << 7); // Clear global interrupt enable bit
  m = my_timer2_millis;
  MY_SREG = oldSREG;    // Restore global interrupts

  return m;
}

#endif
```

```
#include "myMilli.h" // Includes custom myMilli() and Timer 2 ISR

// --- Hardware Pin Definitions ---
const int LED_1 = 2;
const int LED_2 = 3;
const int BUTTON_PIN = 4; // Single-pole, momentary-on button
const int MOTOR_PIN = 5;  // PWM output pin for motor

// --- Serial & LED Task Variables ---
unsigned long interval1 = 0;
```

```
unsigned long interval2 = 0;
bool led1State = LOW;
bool led2State = LOW;
unsigned long previousMillis1 = 0;
unsigned long previousMillis2 = 0;
enum State { WAITING_FOR_LED, WAITING_FOR_INTERVAL };
State currentState = WAITING_FOR_LED;
int selectedLED = 0;

// --- Motor & Button Task Variables ---
// C9 & HL.15 requires: OFF -> LOW -> MED -> HIGH -> MED -> LOW -> OFF
enum MotorState { OFF_1, LOW_1, MED_1, HIGH_1, MED_2, LOW_2 };
MotorState currentMotorState = OFF_1;

int lastButtonState = LOW;
int buttonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;

// --- TASK 1: Handle Serial Monitor Inputs ---
void task_SerialMonitor(void) {
  if (Serial.available() > 0) {
    String input = Serial.readStringUntil('\n');
    input.trim();
    if (input.length() > 0) {
      int value = input.toInt();
      if (currentState == WAITING_FOR_LED) {
        if (value == 1 || value == 2) {
          selectedLED = value;
          currentState = WAITING_FOR_INTERVAL;
```

```
          Serial.println("What interval (in msec)?");
        } else {
          Serial.println("Invalid LED. Please enter 1 or 2.");
          Serial.println("What LED? (1 or 2)");
        }
      }
      else if (currentState == WAITING_FOR_INTERVAL) {
        if (value < 0) {
          Serial.println("Invalid interval.");
        } else {
          unsigned long interval = (unsigned long)value;
          if (selectedLED == 1) {
            interval1 = interval;
            previousMillis1 = myMilli();
            led1State = LOW;
            digitalWrite(LED_1, LOW);
            Serial.print("LED 1 set to blink at "); Serial.print(interval1);
Serial.println(" ms");
          } else if (selectedLED == 2) {
            interval2 = interval;
            previousMillis2 = myMilli();
            led2State = LOW;
            digitalWrite(LED_2, LOW);
            Serial.print("LED 2 set to blink at "); Serial.print(interval2);
Serial.println(" ms");
          }
        }
        currentState = WAITING_FOR_LED;
        Serial.println("What LED? (1 or 2)");
      }
    }
```

```
  }
}

// --- TASK 2: Control LED 1 Timing ---
void task_LED1(void) {
  if (interval1 > 0) {
    unsigned long currentMillis = myMilli();
    if (currentMillis - previousMillis1 >= interval1) {
      previousMillis1 = currentMillis;
      led1State = !led1State;
      digitalWrite(LED_1, led1State);
    }
  } else {
    led1State = LOW;
    digitalWrite(LED_1, LOW);
  }
}

// --- TASK 3: Control LED 2 Timing ---
void task_LED2(void) {
  if (interval2 > 0) {
    unsigned long currentMillis = myMilli();
    if (currentMillis - previousMillis2 >= interval2) {
      previousMillis2 = currentMillis;
      led2State = !led2State;
      digitalWrite(LED_2, led2State);
    }
  } else {
    led2State = LOW;
    digitalWrite(LED_2, LOW);
```

```
  }
}

// --- TASK 4: Non-Blocking Button Polling & State Machine ---
void task_Button(void) {
  int reading = digitalRead(BUTTON_PIN);

  if (reading != lastButtonState) {
    lastDebounceTime = myMilli();
  }

  if ((myMilli() - lastDebounceTime) > debounceDelay) {
    if (reading != buttonState) {
      buttonState = reading;

      // Advance state machine on button press
      if (buttonState == HIGH) {
        switch (currentMotorState) {
          case OFF_1:  currentMotorState = LOW_1;  break;
          case LOW_1:  currentMotorState = MED_1;  break;
          case MED_1:  currentMotorState = HIGH_1; break;
          case HIGH_1: currentMotorState = MED_2;  break;
          case MED_2:  currentMotorState = LOW_2;  break;
          case LOW_2:  currentMotorState = OFF_1;  break;
        }
        Serial.print("Fan State Changed: ");
        Serial.println(currentMotorState);
      }
    }
  }
```

```
  lastButtonState = reading;
}

// --- TASK 5: PWM Motor Controller ---
void task_Motor(void) {
  int pwmValue = 0;

  // Set PWM values based on justified inference for CFM/dB requirements
  switch (currentMotorState) {
    case OFF_1: pwmValue = 0;   break;
    case LOW_1: pwmValue = 90;  break;
    case MED_1: pwmValue = 150; break;
    case HIGH_1:pwmValue = 255; break;
    case MED_2: pwmValue = 150; break;
    case LOW_2: pwmValue = 90;  break;
  }

  analogWrite(MOTOR_PIN, pwmValue);
}

// --- CYCLIC EXECUTIVE SETUP & SCHEDULER ---
typedef void (*TaskFunction_t)(void);

TaskFunction_t taskTable[] = {
  task_SerialMonitor,
  task_LED1,
  task_LED2,
  task_Button,
  task_Motor
};
```

```
const int NUM_TASKS = sizeof(taskTable) / sizeof(taskTable[0]);


void setup() {
  pinMode(LED_1, OUTPUT);
  pinMode(LED_2, OUTPUT);
  pinMode(MOTOR_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT); // Assumes external pull-down resistor


  digitalWrite(LED_1, LOW);
  digitalWrite(LED_2, LOW);
  analogWrite(MOTOR_PIN, 0);


  // Custom Timer 2 Configuration (from myMilli.h)
  MY_TCCR2A = 0x03;
  MY_TCCR2B = 0x04;
  MY_TCNT2  = 0x00;
  MY_TIMSK2 = 0x01;
  MY_SREG  |= 0x80;


  Serial.begin(9600);
  Serial.println("System Ready: LEDs and Motor Active.");
  Serial.println("What LED? (1 or 2)");
}


void loop() {
  for (int i = 0; i < NUM_TASKS; i++) {
    taskTable[i]();
  }
}
```

END OF DOCUMENT